Accessible AI for All

M2 Mid-Submission Update

Branch: `main`
Due: Oct 11, 2026 | 4:00 AM
Path: deliverables/M2.pdf

1. Current Approach & Changes Since M1

> **Tech Stack & Setup:** We are building a clean, web-based app using standard web tech (HTML/CSS) along with Python and Streamlit. Everything on the backend is hooked up directly to our model APIs so data flows smoothly across the entire system.

- **Frontend Status:** Built using standard web development practices (HTML/CSS) and integrated right into Streamlit to give users an easy-to-use interface.
- **Backend Work:** Written entirely in Python. It handles all our business logic, processes inputs, manages session states, and communicates with external services.
- **Connected APIs:** All APIs are linked up to the backend. User actions now trigger backend logic, fire API calls, and update the UI live without full reloads.
- **What Changed Since M1:** Back in M1, we only had the UI mockups ready. Since then, we built out the complete backend, connected all APIs, and pushed the working codebase to `main`.

2. Experiments Run So Far & Results

We ran a few initial tests on our backend setup and API connections to make sure everything runs smoothly during interactive use:

Focus Area	What We Tested	Main Findings	Status
API Response Times	Ran 50 sample API requests through our Python backend to check speed.	Average total latency stayed under **450 ms**, keeping UI interactions snappy.	Working
Session & State Management	Tested keeping user state intact across multi-step inputs in Streamlit.	**100% data retention** across state updates with no accidental resets.	Working
Frontend Integration	Checked if dynamic backend responses rendered properly on the interface.	The UI updated immediately upon receiving data with **no dropped updates**.	Working

3. Progress on Testing

Here is how we are making sure our code stays solid and reliable before final submission:

- **Unit Testing (Backend):** Set up test scripts using Python's standard `pytest` library to verify backend helper functions, data formats, and logic modules.
- **Edge-Case Handling:** Tested how the app handles missing inputs, empty responses, or invalid parameters so the app shows friendly warnings in Streamlit instead of crashing.
- **API Route Checks:** Verified that network connections return expected status codes (`200 OK`, `4xx`, `5xx`) and added retry/timeout checks in Python.
- **End-to-End Walkthroughs:** Conducted end-to-end tests starting from user clicks on the frontend all the way down to backend API calls and view updates.

4. Plan for the Remaining Time (Roadmap to M3)

Now that the backend is finished, APIs are hooked up, and code is merged onto `main`, here is our plan for the remaining project time:

Phase 1: Polish & Speed

- Add Streamlit caching (`@st.cache_data`) to prevent duplicate API hits.
- Touch up HTML/CSS styles for cleaner visuals and accessibility.
- Clean up backend code structure.

Phase 2: Stress & Edge Tests

- Test how the app holds up under concurrent user sessions.
- Wrap up remaining error logs and fallback states.
- Run a final accessibility check.

Phase 3: Final Delivery

- Freeze code on the `main` branch.
- Put together user guides and demo videos.
- Finalize all M3 submission deliverables.

> **Repository Note:** All code developed so far—including our Python backend scripts, API wrappers, Streamlit layout integrations, and test cases—has been committed and pushed to the `main` branch.